

CPSC 433/533

Project 3 - Domain Name System

1 Problem Statement

The goal of this project is for you to learn to implement application layer protocols on the top of the IP and transport layers that we visited in the previous two projects. You will implement an algorithm for domain name system (DNS) which hierarchically finds IP address for a given domain name. We will use the routing simulator that we used for the first project as a framework which delivers packets between DNS name servers that run your DNS resolution algorithm.

The task at hand is to implement a DNS resolution algorithm to provide scalable, reliable, and performant DNS service. A DNS client generates and sends a DNS query to a DNS resolver which is a special DNS name server that will perform DNS lookups. DNS name servers, including the resolver, parse the received DNS query and search through their local DNS records. If it is needed, the servers forward the DNS query to other DNS name servers. Once the IP address of the queried domain name is resolved, the result will be returned to the DNS resolver then finally to the DNS client.

2 Simulation Environment

We will use a similar simulation environment we used in the first project.

Below we describe only the classes you will need to implement. Your DNS name server, `DNSServer`, will extend the `UDPHost` (which is built on top of the `HostEntity` class defined in `sim/basics.py`; we visited this class in the first project). Each `DNSServer` can have two kinds of DNS records: a) host of the domain (A-type; a mapping from a hostname string to the IP address of host server) and b) DNS name server (NS-type; for simplicity, we use a single NS-type entry to represent a mapping from the domain name directly to the *IP address* of the `DNSServer`). The `DNSServer` class has eight functions that will be relevant to you:

```
class DNSServer(UDPHost)
    handle_dns(self, packet)
        Automatically called by UDPHost when it receives a packet
        via a UDP port dedicated to DNS.
        packet - a Packet (or subclass).
        You definitely want to override this function, in class DNSServer.

    send_dns_data(self, packet, address)
        Sends packet to the destination.
        packet - a Packet (or subclass).
        address - an IP address of the destination (refer to the next section for more details).
        Do not override this function.

    add_dns(self, domain, address, dns_ttl)
        Adds the DNS record from domain to address (A-type).
        domain - a host name in str, such as "my.webpage.example.com".
        address - an IP address of the host that provides service via the domain.
        dns_ttl - a float variable for time-to-live in second (for caching).
        You definitely want to override this function, in class DNSServer.
```

```

del_dns(self, domain)
    Deletes the DNS record for domain (A-type).
    domain - a host name of the record we want to remove.
    You definitely want to override this function, in class DNSServer.

add_dns_server(self, domain, address)
    Adds the address of DNS name server (address) for domain (NS-type).
    domain - a domain in str such as "example.com".
    address - an address of the DNS name server.
    You definitely want to override this function, in class DNSServer.

del_dns_server(self, domain, address)
    Deletes the DNS name server specified by address for the given domain
    (NS-type; delete only the exactly matched entry).
    domain - a domain of the record we want to remove.
    address - an address of the DNS name server.
    You definitely want to override this function, in class DNSServer.

get_dns(self, domain)
    Returns a list of the addresses of the DNS name servers for the given domain
    or the address of the matched host server serving the domain (from local DNS records).
    If there is no record corresponding to the domain, it must return empty list.
    The result must be always a list containing zero, one, or multiple elements.
    Local DNS records should include cached records as well.
    domain - the queried domain in str.
    You definitely want to override this function, in class DNSServer.

set_recursive(self, is_support_recursive)
    Updates a local variable, self.is_support_recursive.
    is_support_recursive - a boolean value (True or False).
    Do not override this function.

self.is_support_recursive
    Only if this value is set to True,
    the DNS name server will perform recursive DNS lookup.
    Do not change variable in class DNSServer, other than inside set_recursive().

```

In addition, you need to take a look at the class named `DNSResolver`, which extends `DNSServer`. The only difference between `DNSResolver` and `DNSServer` is that `self.is_support_recursive` is always set to `True` in `DNSResolver` and will not be updated by `DNSResolver.set_recursive`. *Do not override class `DNSResolver`.*

The `DNSPayload` class extends the `Packet` class in the first project (located in `sim/basics.py`). In addition to the fields inherited from `Packet`, we add four fields as an application payload:

```

class DNSPayload(Packet)
    self.opcode
        An opcode in str class, which must be either "request", "response_found_host",
        "response_found_dns_server", or "response_not_found".
    self.request
        It will contain a domain for the DNS request as str.
        This field must not be modified in DNSServer.
    self.response
        It must be a list of addresses as list of str objects

```

```

        (which contains one or multiple addresses of DNS name servers
         or a single address of the host).
self.dns_ttl
    Time-to-live value for the found DNS record in float.
    This value must be set as: the current time (time.time()) + dns_ttl (as specified
    in DNSServer.add_dns()) at the time when this response is generated
    (i.e., it must not be updated afterwards).

```

3 DNSServer Specification

Write a `DNSServer` class which inherits from the `UDPHost` class. The main function that you will need to write is `handle_dns()` which will deal with `DNSPayload`. You will also need to write five utility functions, `add_dns()`, `del_dns()`, `add_dns_server()`, `del_dns_server()`, and `get_dns()`, with your own representation for the DNS records (feel free to add more member functions to `DNSServer`).

`DNSPayloads` are initially sent by a DNS client with “request” opcode and the host name in its request. Inside `handle_dns()`, you must handle these packets properly. You need to call `send_dns_data()` to send the response back to the sender or to forward the packet to the next DNS name server (you can read IP address of sender and receiver of `Packet` via `Packet.src.ip_addr` and `Packet.dst.ip_addr`, respectively; do not directly modify the addresses). Note that the client can send more than one request in a given time; however, we will manage it to do not send multiple requests for the same domain name (until the client receives a response for the previous request having the same domain name).

During the processing of `handle_dns`, `DNSServers` **must not** generate new packets; instead, they must update and forward the received packet to other DNS name servers or the requester. Do not override the information carried by underlying `Packet` class (i.e., lower layer fields than `DNSPayload` or application layer). Also, do not override either `send_dns_data()` or `UDPHost` class. We will not inject packet drop or data corruption. In specific, your implementation should perform the following functions:

- **DNS configuration.** DNS records must be configured properly through the utility functions, so that any later DNS queries/requests must be processed following the records. Note that “foo.bar.com” should be considered as a sub-domain of “bar.com”. We will not use other formats such as “*.bar.com”. For simplicity, we assume that there is no root name server and the address of the top name servers (e.g. for “com”) is directly given to the DNS resolver. You can assume that a DNS server will not be requested to have the same string for both DNS name server (`add_dns_server()`) or hostdomain (by `add_dns()`).
- **Runtime updates.** If someone tries to add an A-Type DNS record which is already populated in the DNS name server (via `add_dns()`), such addition must be treated as an update request and the record must be overridden. On the other hand, if someone tries to add DNS name server for the same domain (an NS-type record via `add_dns_server()`), it must be treated as a request for adding replicated DNS name server.
- **DNS lookup.** Your DNS name server must properly parse the received DNS request and send a correct response. If the domain name is invalid, the DNS name server must return a response of which the *opcode* is set to “response_not_found”. You can assume that each DNS server will have DNS records for the same or more detailed domain name (there will be no DNS look-up chain across DNS servers such as “example.com” - “web.example.com” - “example.com”).
- **Iterative and recursive lookups.** Iterative and recursive DNS queries must be handled properly based on `is_support_recursive` in `DNSServer`. If `is_support_recursive` is `True`, the `DNSServer` must perform recursive DNS lookups. Otherwise, the `DNSServer` must iteratively search through DNS records. Recursive DNS lookup means that the `DNSServer` can send DNS requests to other DNS name servers until it figures out the result (i.e, found or not found). On the other hand, in case of iterative DNS lookup, the `DNSServer` searches only its local DNS records and immediately sends a response back to the sender.

- **Availability.** Note that there can be replicated DNS name servers for better availability. The DNS name server must be able to check all DNS name servers matched for the requested domain.
- **Caching.** Intermediate DNS name servers may cache DNS records. The cached record must be invalidated if the current time observed in a DNS name server (i.e., return value of `time.time()`¹) is later than `dns_ttl` of the cached DNS record (e.g., check and remove DNS records every time a DNS name server receives a new packet.). Everytime a DNS name server see a packet for the cached DNS record but having newer `dns_ttl`, it must update `dns_ttl` of the *cached DNS record*.

4 Tips

- Check the public test cases carefully. They would help you get a sense of the project environment such as a string format of domains and a basic topology between the client, DNS resolver, and other DNS name servers.
- Check out the simulator guide provided with *Project 1* to see how to run your tests. In this project, we provide a sample compatibility test and a simple functionality test.
- Your DNS name servers should only have reactive behaviors and their behaviors should be deterministic. Avoid using things like random number generators.

5 Grading

Your DNS name server will be run against a number of test cases, and we will evaluate it based on the following aspects:

Does it work? (80 pts) Tests in this category evaluate the correctness and basic functionalities of your `DNSServer`. For example, the protocol must be able to find the correct address for the requested domain name according to the up-to-date DNS records (you will get penalty if your DNS name server generates too many extra packets during DNS lookups across DNS name servers).

Caching (20 pts) For better performance and shorter response time, your DNS name server may cache DNS records. Note that the server must cache both positive and negative responses (of which the opcode is “*response_found*” or “*response_not_found*”, respectively). For negative responses, its time-to-live must set to **60 seconds**. Any outdated records must be ignored and removed (you will get penalty if your DNS name server simply adds newly cached DNS records without cleaning up the outdated records).

The public tests provided with the starter code are only a fraction of the total tests that we will run your code against. It is your responsibility to implement other tests to make sure your implementation is correct. Note that we will use **python3.9.10** to run the tests. Also, the starter code does not contain `dv_router.py`. Therefore, you will need to copy and use your `dv_router.py` implementation from Project 1. For grading, we will use a private/working version of `dv_router.py`.

To run the public tests:

```
ruby public_tests.rb
```

Warning: Keep in mind that we are accepting only one version of your implementation. It’s easier to screw up the 80 pts worth of basic tests if you implement the last two tasks close to the deadline, in what you might think is a caffeine-induced blaze of glory. Start early, write lots of incremental tests, and make sure new features don’t cause earlier tests to fail.

¹<https://docs.python.org/3/library/time.html>

6 README

You must also supply a `README.txt` file along with your solution. This should contain:

1. What challenges did you face in implementing your DNS name server?
2. Name a feature NOT specified in this specification that would improve your DNS name server.
3. Specify if your code employs DNS caching. If you have implemented it, describe what additional considerations need to be taken into.
4. Number of hours it took you to complete the assignment (be honest).

Note that your submission is incomplete without the `README.txt`.

7 Downloads

Check the Canvas assignment for the latest copy of the project code.

8 What to Turn In

Turn in a `.tar` file with the following format:

```
project3-lastname-firstname.tar
```

(e.g. `project3-einstein-albert.tar`). Make sure to use the firstname and lastname provided in the class roster. The `.tar` file should contain `dns_server.py` that implements the `DNSServer` outlined above. It should also include your `README.txt` file. Use the following command to tar your archive:

```
tar -cf project3-lastname-firstname.tar dns_server.py README.txt
```

9 Cheating and Other Rules

You should not touch the simulator code itself (particularly code in `sim/core.py`). We are aware that Python is self-modifying and therefore you could write code that rewrites the simulator. *You will receive zero credit for turning in a solution that modifies the simulator itself. Don't do it.*

You may not share code with any classmates, including your test code. You may discuss the assignment requirements or your solutions (e.g., what data structures were used to store routing tables, test scenarios) — *away from a computer and without sharing code* — but you should not discuss the detailed nature of your solution (e.g., what algorithm was used to compute the routing table). Also, don't put your code in public repository. Assignments suspected of cheating or forgery will be handled according to the Student Code of Conduct.² Apparently 23% of academic misconduct cases at a certain junior university are in Computer Science,³ but we expect you *all* to uphold high academic integrity and pride in doing *your own work*.

²<http://catalog.yale.edu/undergraduate-regulations/regulations/academic-dishonesty/>

³http://www.pcworld.com/article/194486/why_computer_science_students_cheat.html